

CSCE 606 SOFTWARE ENGINEERING · MODULE 07 · LECTURE NOTES

Refactoring and Technical Debt

Changing structure without changing behaviour, and deciding when to.

MODULE OUTCOMES

- Define refactoring precisely and work in safe steps.
- Recognise common code smells and their corresponding refactorings.
- Treat technical debt as a financial decision, including deliberate debt.
- Decide when to refactor and when to leave it.
- Explain why rewrites usually fail.

1 What refactoring is

THE DEFINITION, AND WHY PRECISION MATTERS

A refactoring is a change to the internal structure of software that makes it easier to understand and cheaper to modify, without changing its observable behaviour.

If the observable behaviour changes, it is not a refactoring. It is a change, and it requires the testing and review that any behavioural change requires.

The precision licenses the entire technique: because behaviour is unchanged by definition, a test suite that passed before the refactoring must pass after it. The tests become a mechanical check on the transformation.

Never combine a refactoring with a behavioural change in one commit. When something breaks — and it will — you need to know which of the two caused it, and a mixed commit makes that impossible. It also makes review far harder: a reviewer can skim a pure rename and must read a behavioural change carefully.

1.1 The method

- 1 **Ensure tests exist and pass.** If there are none, write characterisation tests first (Module 06).
- 2 **Make one small change.** Extract a function. Rename a variable. Move a method. One transformation.
- 3 **Run the tests.** If they fail, **undo** — do not debug. The step was too large or the transformation was wrong; either way, returning to a known-good state costs seconds and debugging costs hours.
- 4 **Commit.** Then repeat.

The discipline is in the step size. A large refactoring that breaks leaves you with a working state and a broken state and no way to bisect between them. Fifty small refactorings leave fifty checkpoints, each

trivially revertible, and git bisect will find the problem in six steps.

1.2 Smells

Smell	What it suggests	Usual refactoring
Long function	Several responsibilities in one place.	Extract function. Each extracted piece gets a name, which is most of the benefit.
Long parameter list	A missing type — several parameters that always travel together.	Introduce parameter object.
Duplicated knowledge	One fact represented in two places (Module 03's DRY test).	Extract and call.
Feature envy	A method uses another class's data more than its own.	Move method to where the data is.
Shotgun surgery	One conceptual change requires edits in many files.	A module is missing. This is Module 03's strongest signal.
Primitive obsession	Domain concepts represented as strings and integers.	Introduce a type. EmailAddress rather than str makes a class of bugs impossible.

A SMELL IS A HYPOTHESIS

A long function is often doing several things. It is sometimes a single sequential process that reads more clearly written out than split into twelve single-line functions, each of which must be located and read.

A long parameter list often conceals a missing type. It sometimes reflects a function that genuinely requires seven independent values.

Smells tell you where to look, not what to do. Treated as rules and enforced by tooling, they produce codebases that satisfy a linter and are harder to read than what they replaced — which is a real and common failure.

The test is always Module 01's: *does this change reduce the cost of the next change?* If the answer is 'it satisfies a guideline', the answer is no.

2 Technical debt

Ward Cunningham's metaphor is more precise than its common use suggests, and the original formulation is worth recovering.

THE ORIGINAL ARGUMENT

Shipping code that does not yet fully reflect your understanding of the problem is like **borrowing money**: you obtain value now — you ship, you learn from users — and you pay interest later in the form of harder changes.

Interest is the additional cost imposed on every subsequent change by the shortcut. **Principal** is the cost of fixing it properly.

Borrowing is frequently correct. Shipping six months earlier may be worth years of interest, particularly when shipping is how you discover whether the product should exist.

Cunningham's point was specifically about *learning*: you write code reflecting your current understanding, that understanding improves, and the gap between the code and your improved understanding is the debt. It was never a licence for sloppiness, and it is frequently misquoted as one.

	Deliberate	Inadvertent
Prudent	'We must ship now; we will fix this after launch.' A legitimate decision , provided it is recorded.	'Now that it is built, we understand how it should have been done.' Unavoidable and healthy — this is Cunningham's original case.
Reckless	'We do not have time for design.' A decision, and a bad one.	'What is a module?' Debt incurred through ignorance, which is the only quadrant that is simply a problem.

RECORD THE DEBT OR IT WILL NEVER BE REPAYED

Deliberate prudent debt is good engineering. **Undocumented** deliberate debt is indistinguishable, six months later, from incompetence.

When you take a shortcut knowingly, write down what you did, why, and what it will cost — a comment, an issue, an ADR. The form matters less than the existence.

With that record, the debt can be reviewed, prioritised against feature work, and repaid when the accumulated interest justifies it. It becomes a managed liability.

Without it, a future maintainer encountering the odd code cannot tell whether it is a deliberate trade-off or a mistake — so they will work around it rather than fix it, and the interest compounds indefinitely.

3 When to refactor

Refactor	Do not refactor
Immediately before changing code. 'Make the change easy, then make the easy change' (Kent Beck). The restructuring is justified by a change you are definitely making.	On a scheduled cleanup sprint with no specific change in view. You will restructure toward imagined future requirements, which is speculation, and speculation about requirements is usually wrong.
Immediately after changing code , while you still understand it and the context is loaded.	Code you are not going to change. Ugly, stable, working code accrues no interest.
When the same awkwardness has cost you three times. Three is evidence; once is noise.	To satisfy a style preference or to make a linter quieter.
When a bug's root cause is structural — fixing the structure prevents the next three bugs.	Across a whole codebase at once. Large refactorings are unreviewable and conflict with everything in flight.

PRIORITISE BY CHANGE FREQUENCY, NOT BY UGLINESS

Technical debt accrues interest only on code that is actually modified. A module untouched for four years is paying no interest whatever it looks like, and refactoring it is pure cost.

Version control history tells you exactly where the interest is: which files change most often, and which changes take longest. That data is free and nobody looks at it.

The intersection of 'changes frequently' and 'is hard to change' is where all the value is. Everything outside that intersection is aesthetics with a cost attached.

This is also why 'the whole codebase needs cleaning up' is almost never the right plan, and why it almost never gets approved — correctly.

4 Rewrites

The strongest temptation in software engineering is to declare a system beyond saving and start again. It is usually a mistake, for reasons that are specific rather than merely cautionary.

- 1 The old code encodes knowledge you no longer have.** Every strange conditional is, in all likelihood, a bug somebody found and fixed. The code is ugly precisely because reality is, and the accumulated special cases are the record of five years of encounters with reality.
- 2 You will reintroduce every one of those bugs**, individually, and rediscover each over the following years — this time with users watching.
- 3 The target moves.** The old system does not stop evolving while you rebuild, so you are implementing a specification that is being rewritten as you work.
- 4 No value is delivered until it is complete**, and it will take substantially longer than estimated — so there is a long period during which two systems must be maintained and nothing has improved.

THE STRANGLER FIG

Martin Fowler's alternative, named after the plant that grows around a tree and gradually replaces it.

Build the new system *around* the old one. Route all traffic through a facade. Implement one capability in the new system and divert that traffic to it. Delete the corresponding old code. Repeat.

Every step delivers value, every step is revertible, and the system works throughout. There is never a big-bang cutover and never a period where nothing ships.

It is slower in principle and faster in practice, and it is the difference between a migration that completes and one that is abandoned two years in with both systems running.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Martin Fowler — Refactoring catalogue	Every refactoring with its mechanics. The reference to keep open while working.
Fowler — Technical Debt Quadrant	The deliberate/inadvertent and prudent/reckless distinction of §2.
Ward Cunningham — Debt Metaphor (free video)	Cunningham explaining what he actually meant, and correcting the common misuse. Five minutes.
Joel Spolsky — Things You Should Never Do, Part I	The canonical argument against rewrites, with the Netscape case. Twenty years old and still correct.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Take a long function and extract sub-functions in small steps, running tests between each. Commit after every step. Then review the commit history and judge whether each step was independently sensible.
- 2 Use version control history to find the files in a project that change most frequently. Cross-reference against the files you find hardest to read. Refactor the intersection.
- 3 Find a piece of deliberate technical debt in your own code. Write the record you should have written: what, why, what it costs, what would trigger repaying it.
- 4 Take a function with primitive obsession — strings and integers standing in for domain concepts — and introduce types. Count the bugs that become impossible to express.

- 5 Find an instance of shotgun surgery using commit history: files consistently changed together. Propose the module that is missing.
- 6 Attempt a refactoring in one large step, break something, and experience the difficulty of locating the cause. Then revert and do it in small steps.
- 7 Find a system you use that was rewritten from scratch. Research what happened and write a paragraph assessing it against §4's four reasons.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Define refactoring precisely, and say why the precision licenses the technique.
- 2 Why should a refactoring never be combined with a behavioural change in one commit?
- 3 Why undo rather than debug when tests fail mid-refactoring?
- 4 Why is a code smell a hypothesis rather than a rule?
- 5 What are interest and principal in the technical debt metaphor, and when is borrowing correct?
- 6 Why does undocumented deliberate debt never get repaid?
- 7 Give two times to refactor and two times not to.
- 8 Give three specific reasons rewrites fail, and name the alternative.